



Tribhuvan University
Institute of Science and Technology

A Midterm Project Report On
“Custom VPN with secure SSL/TLS tunneling”

In partial fulfillment of the requirements
For the Bachelors of Science in Computer Science and Information Technology

Submitted by:

Deepan Paudel

Ristika Maharjan

Sulav Adhikari

Asian College of Higher Studies

Ekantakuna, Lalitpur

Submitted to:

Department of Computer Science and Information Technology

Under the supervision of:

Mr.

April, 2026

Tribhuvan University
Institute of Science and Technology



SUPERVISOR'S RECOMMENDATION

I hereby recommend that this project prepared by **Mr. Deepan Paudel, Mr. Ristika Maharjan, and Mr. Sulav Adhikari** under the supervision of entitled **“Custom VPN with secure SSL/TLS tunneling”** in partial fulfillment of the requirement for the degree of Bachelors of Computer Science and Information Technology has been processed for the evaluation

.....

Project Supervisor

Department of Computer Science and Information Technology

Tribhuvan University
Institute of Science and Technology



LETTER OF APPROVAL

This is to certify the project prepared by Deepan Paudel, Ristika Maharjan, and Sulav Adhikari entitled “**Custom VPN with secure SSL/TLS tunneling**” in partial fulfillment of the requirement for the degree of Bachelors of Computer Science and Information Technology has been evaluated. In our opinion it is satisfactory in the scope and quality as a project for the required degree.

.....

Project Supervisor

Department of Computer Science and Information Technology

ACKNOWLEDGEMENT

We would like to express our sincerest gratitude to our project supervisor **Mr.**, for his invaluable guidance, encouragement, and constructive feedback throughout the development of this project. His continuous support and insightful suggestions have been instrumental in shaping this work into its final form.

We are also deeply thankful to my friends, classmates, and seniors who provided both direct and indirect support during the course of this project. Their cooperation, motivation, and timely assistance played a significant role in helping us overcome challenges and complete the work within the given time frame.

Lastly, we would like to thank our classmates, family, and friends for their constant motivation and assistance during the preparation of this compilation. Without their support, this work would not have been possible.

Sincerely,

Deepan Paudel

Ristika Maharjan

Sulav Adhikari

ABSTRACT

Digital communication over public networks demands secure, transparent, and controllable data transmission mechanisms. Existing Virtual Private Network (VPN) solutions present a persistent trade-off: commercial products are easy to deploy but lack transparency and require trust in third-party providers, while open-source alternatives such as OpenVPN and WireGuard introduce complex dependencies, kernel-level requirements, and opaque implementation layers. Existing Layer 3 VPN architectures expose authenticated users to the entire internal network subnet, creating lateral movement risk that enterprise ZTNA solutions address at prohibitive per-user licensing cost.

This report defines the design and implementation of a custom application-layer VPN built entirely in Python 3.11 using no external packages, no kernel modules, and no elevated privileges. The system implements a complete eight-step session lifecycle secured through TLS 1.2 or higher with Elliptic Curve Diffie-Hellman Ephemeral (ECDHE) key exchange enforcing Perfect Forward Secrecy (PFS), AES-256-GCM data encryption, and PBKDF2-HMAC-SHA256 user authentication with brute-force lockout. The broader platform extends the core with a FastAPI management API and a React administration dashboard, though this report focuses on the VPN core layer.

Keywords: *VPN, ZTNA, SSL/TLS, ECDHE, PBKDF2, application-layer tunneling, Python, network security*

TABLE OF CONTENTS

SUPERVISOR’S RECOMMENDATION	I
LETTER OF APPROVAL	II
ACKNOWLEDGEMENT.....	III
ABSTRACT.....	IV
LIST OF FIGURES	VII
LIST OF TABLES.....	VIII
LIST OF ABBEVIATIONS	IX
CHAPTER 1 : INTRODUCTION.....	1
1.1 Introduction.....	1
1.2 Problem Statement.....	2
1.3 Objectives.....	3
1.4 Scope and Limitation	3
1.5 Development Methodology.....	5
1.6 Report Organization	5
CHAPTER 2 : LITERATURE REVIEW	7
2.1 Background Study	7
2.2 Literature Review	8
CHAPTER 3 : SYSTEM ANALYSIS.....	12
3.1 System Analysis	12
3.1.1 Requirement Analysis	12
3.1.2 Feasibility Analysis.....	14
3.1.3 Object-Oriented Analysis	16
CHAPTER 4 : SYSTEM DESIGN	21
4.1 Object Oriented System Design	21
4.1.1 Refinement of Class diagram.....	21
4.1.2 Refinement of Object diagram	22

4.1.3 Refinement of State diagram	23
4.1.4 Refinement of Sequence Diagram	24
4.1.5 Refinement of Activity Diagram	25
4.1.6 Component Diagram	26
4.1.7 Deployment Diagram	26
4.2 Algorithm Details	27
CHAPTER 5 : IMPLEMENTATION AND TESTING	33
5.1 Implementation	33
5.1.1 Tools Used	33
5.1.2 Implementation Details of Modules	34
5.2 Testing	37
5.2.1 Test Cases for Unit Testing	37
5.2.2 Test Cases for System Testing	38
5.3 Result Analysis	39
CHAPTER 6 : CONCLUSION.....	41
6.1 Conclusion	41
6.2 Future Recommendation	41
REFERENCES	43
APPENDICES	46

LIST OF FIGURES

Figure 3. 1: Use Case Diagram of VPN	13
Figure 3. 2: Gantt Chart of VPN System	16
Figure 3. 3: Class Diagram of VPN system.....	Error! Bookmark not defined.
Figure 3. 4: Object Diagram of VPN System.....	17
Figure 3. 5: State Diagram of VPN System	18
Figure 3. 6: Sequence Diagram of the VPN system	19
Figure 3. 7: Activity Diagram of VPN System.....	20
Figure 4. 1: Refinement of Class Diagram of VPN System.....	21
Figure 4. 2: Refinement of Object Diagram of VPN System	22
Figure 4. 3: Refinement of State diagram of VPN System	23
Figure 4. 4: Refinement of Sequence Diagram of VPN System	24
Figure 4. 5: Refinement of Activity Diagram of VPN System.....	25
Figure 4. 6: Component Diagram of VPN System	26
Figure 4. 7: Deployment Diagram of VPN System	26

LIST OF TABLES

Table 5. 1: Programming Tools used in VPN system	33
Table 5. 2: Framework and Libraries used in VPN system	33
Table 5. 3: Development and Testing Tools used in VPN system	34
Table 5. 4: Table for test cases of Unit Testing.....	37
Table 5. 5: : Table for test cases of System Testing.....	38

LIST OF ABBREVIATIONS

AES	Advanced Encryption Standard
API	Application Programming Interface
CVE	Common Vulnerabilities and Exposures
ECDHE	Elliptic Curve Diffie-Hellman Ephemeral
HMAC	Hash-based Message Authentication Code
IPSec	Internet Protocol Security
L2TP	Layer Two Tunneling Protocol
LDAP	Lightweight Directory Access Protocol
MFA	Multi-Factor Authentication
ORM	Object-Relational Mapping
PBKDF2	Password-Based Key Derivation Function 2
PFS	Perfect Forward Secrecy
PKI	Public Key Infrastructure
PPTP	Point-to-Point Tunneling Protocol
PSTN	Public Switched Telephone Network
REST	Representational State Transfer
SHA-256	Secure Hash Algorithm 256-bit
SOC	Security Operations Center
SPA	Single Page Application
SSL	Secure Sockets Layer
TLS	Transport Layer Security
VPN	Virtual Private Network
ZTNA	Zero Trust Network Access

CHAPTER 1 : INTRODUCTION

1.1 Introduction

Digital Communication supports nearly every aspect of business, governance, and personal interaction, making secure data transmission over public networks a necessity. Virtual Private Network (VPNs) address these needs by creating encrypted tunnels, allowing the users to exchange information while maintaining confidentiality, integrity, and authentication.

A VPN is a technology that uses tunneling, encryption, authentication, authorization, and access control to securely transfer data across shared networks such as the Internet, Intranet, or Extranet, amidst of behaving like a private network [1]. A virtual private network extends like a secure “tunnel” to connect a private network across the internet (public network) allowing users to send and receive data through encrypted tunnels as if their computing devices were directly connected to the private network [2] [3]. VPN evolved from the need to provide private like network over public by the Public Switched Telephone Network (PSTN) [1]. In today’s context VPN developed have implemented different protocols, technologies, and solutions. The technologies used can be: IPSec (IP Security protocols), Point-to-Point Tunneling Protocol, Layer Two Tunneling Protocol, or SSL/TLS using different security management technologies [2].

VPN has always been in demand tools for remote access and this hadn’t changed till 2026; they occupy a strange paradox. Client yet requests for VPNs and the market is still expanding [4]. Commercial VPN services such as NordVPN, ExpressVPN, Cisco AnyConnect, and Fortinet FortiGate are easy to setup and can grow as needed, but they don’t clearly show how their security systems work, leading to concerns regarding trust and vendor dependency [4]. This design implicitly trusts any authenticated users with access to the entire internal subnet, creating a lateral movement risk where a single compromised credential can expose all reachable internal services [5].

The severity of this architectural weakness is documented in real-world incidents. Serious vulnerabilities, such as CVE-2019-11510 in Pulse Secure and CVE-2023-3519 CVE-2019-11510 in Pulse Secure and CVE-2023-3519 which allowed attackers to access internal systems and access the credential without even logging in, this happened because the VPN gateway acted as a single high-value target and did not separate the login process from

actual network access. [6] [7]. Open source alternatives such as OpenVPN and WireGuard offer greater auditability but do not resolve the underlying problem: both grant network-layer access upon authentication, rely on firewall rules to restrict lateral movement, and require either kernel module installation or significant configuration expertise to deploy securely [8] [9].

This gap is addressed by Enterprise Zero Trust Network Access (ZTNA) solutions by giving each application its own access control, where users are verified based on their identity, instead of granting access to the entire network. However, the infrastructure complexity and per-user licensing costs of these solutions place them beyond the reach of small to medium organizations, academic institutions, and self-managed infrastructure teams [5]. The result is that organizations unable to adopt enterprise ZTNA remain dependent on the same Layer 3 VPN architecture whose structural weaknesses are well-documented [4] [5].

This project is motivated by this gap. It focuses on implementing a custom SSL/TLS-based application layer on pure Python 3.11, designed specifically for small to medium organizations, academic environments, and restricted deployment contexts where kernel-level access is unavailable and commercial licensing is not feasible. By operating as a socket-to-socket relay at application layer, the system doesn't assign virtual network interface and no internal IP address to authenticated client, making lateral movement architecturally impossible rather than policy-dependent. In succeeding phases, the VPN core is extended with FastAPI-based management API and a React administration dashboard, forming a complete, self-hosted three-tier access management platform.

1.2 Problem Statement

The problem this project addresses is a specific and measurable gap in the access security model of existing VPN solution: traditional VPNs operate at the network layer this grant every authenticated user virtual IP address and implicit access to the entire internal subnet. A single compromised credential exposes whole network to lateral movement, not just the specific service that user requires [1] [2]. This weakness has direct, documented consequences where CVE-2019-11510 (Pulse Secure) [6] and CVE-2023-3519 (Citrix) [7] enabled unauthenticated attackers to compromise enterprise networks precisely because monolithic VPN gateways lack separation between control and data planes [4].

Existing alternatives for Zero Trust such as Enterprise ZTNA products from vendors including Zscaler and Palo Alto Networks solve this problem through enforcement of per application, identity-verified access but introduce significant licensing cost, vendor dependency, and infrastructure requirements that are prohibitive for small to medium organizations, academic institutions, and self-managed teams [5]. WireGuard eliminates the monolithic gateway problem by design but abandons TLS entirely in favor of the Noise protocol framework, requires kernel module installation, and provides no per-service access control at the application level [8]. OpenVPN uses TLS for its control channel but not for end-to-end data encryption, requires external dependencies, and still grants network-layer access upon authentication [9]. Commercial black-box products offer no cryptographic transparency and require unconditional trust in third-party providers, an assumption that Mixon-Baca et al. demonstrated is frequently unjustified [10].

This gap this project is trying to address is specific and bounded. Currently, there is no free, self-hosted solution at application layer that provides services-level access control works without kernel modules, or extra dependencies, and makes all cryptographic decisions auditable. This gap affects small to medium organizations, academic institutions, and infrastructure teams who require secure, controlled remote access but cannot justify the cost or complexity of enterprise ZTNA. The consequence of this gap, as the vulnerability record demonstrates, is continued dependence on an architectural model that grants excessive trust to authenticated users and presents a concentrated, high-value attack surface to adversaries [4] [5] [6] [7].

1.3 Objectives

The objective of this project is to design and implement a custom SSL/TLS application-layer Virtual Private Network (VPN) in Python 3.11 makes lateral movement inherent in network layer architecturally impossible by enforcing per-service, identity-verified access at the protocol level, while remaining self-hosted and deployable without kernel modules, commercial licensing, or external package dependencies.

- To build a secure tunnel that works in application layer with ZTNA principles
- To enforce per-service, per-user access control at the protocol level
- To minimize lateral movement using socket-to-socket relay at the application layer

1.4 Scope and Limitation

This project implements SSL/TLS-style VPN operating at the application layer between a Python client and server. This report focuses on the VPN core layer. The management layers are complete but are outside the scope of this midterm report thus are briefly described in section 1.1 and referenced for completeness. The scopes cover the following:

- The system provides secure remote access to internal network services by establishing an encrypted communication channel over the public network with no virtual network adapter and no internal IP assigned to the client.
- It implements the complete SSL/TLS session lifecycle, including TLS 1.2 or higher handshake, certificate verification using X.509, and secure key exchange using ECDHE.
- The project implements per-service access control at the protocol level, which the server validates against the authenticated user's permissions before any tunnel is established, separating the control plane from the data plane to reduce pre-authentication attack surface
- It supports local port forwarding, enables bidirectional encrypted data transfer, and includes session management features.
- The entire system is developed using pure Python without external dependencies, making it lightweight, transparent, and easy to deploy across platforms.

The following are outside the scope of this prototype and are assumed as future extensions:

- It provides security primarily for data in transit, but does not protect against threats on the client device such as malware or keyloggers.
- The authentication mechanism is limited to username and password, lacking advanced security mechanisms like multi-factor authentication (MFA).
- The system doesn't include load balancing or failover mechanisms; thus, system failure will disrupt all active connections.
- The monitor diagnostics endpoint currently uses a default control secret, which is noted as a security gap; randomized secret generation at startup is identified as a priority future enhancement
- It does not include advanced enterprise features such as intrusion detection, and automated threat response are not included in current implementation.

1.5 Development Methodology

This project follows a qualitative research methodology to understand, design, and implement a custom SSL/TLS-based VPN system. The qualitative approach was chosen as the project focuses on understanding the concept, system behaviors, and security mechanisms rather than numerical data analysis. The development process includes the literature reviews that reviews over the existing VPN technologies, SSL/TLS protocols, authentication mechanisms and tunneling techniques from academic paper, documentation, and industry resources. After that there is understanding on how different VPN solution works and identifying gaps. Comparative study is conducted to evaluate the existing solutions to determine their limitations and differences in implementation. These all act as guide for the design and development of the system.

Through which to ensure the systematic develop and proper organization of tasks the project uses waterfall model of software development lifecycle. The development progresses through sequential phases including the requirement analysis, system design, implementation, testing, deployment and maintenance. It should be noted that the broader project also uses an iterative, phase-based approach for the management layers which are FastAPI-based management API and a React administration dashboard, reflecting the realities of incremental delivery. The Waterfall model is applied to the core layer specifically because its security requirements are fully specified before implementation begins and do not change mid-development.

1.6 Report Organization

The report has been organized into six chapters:

Chapter 1: Introduction, the background of the project has been provided highlighting the necessity of VPN in today's context followed by the problem statement, objectives, scopes and limitation, development methodology, and report organization.

Chapter 2: Literature Reviews, reviews the existing academic research and industry publications on SSL/TLS VPN architecture, TLS 1.2 handshake mechanics, ECDHE key exchange, PBKDF2-based key derivation, session management, and application-layer VPN design.

Chapter 3: System Analysis, presents the complete requirement analysis including the functional and nonfunctional requirements and system constraints as well as feasibility study and the existing system analysis.

Chapter 4: System Design, presents the complete modular structure, configuration scheme, authentication flow design, and all the involved steps.

Chapter 5: Implementation and Testing, provides the detailed description of each module's implementation, followed by the test cases results covering successful session operation, brute-force lockout, session expiry, and abnormal disconnection.

Chapter 6: Conclusion and Future Recommendation, summarizes the project outcomes and confirms the security properties demonstrated and defines the possible ways to upgrade the system.

CHAPTER 2 : LITERATURE REVIEW

2.1 Background Study

While the previous section identified the core problem that network layer VPN architectures grant excessive network-level trust to authenticated users, creating lateral movement risk that existing affordable solutions do not adequately address, this section, reviews the existing research and technologies to understand why such solutions are not available and why the problem persists and what prior work informs this project's design.

The VPN uses tunneling, encryption, authentication, authorization, and access control to securely transfer data across shared infrastructure such as Internet, Intranet, or Extranet, while behaving like a private network [1]. The concept of VPN is straight forward, it wraps data in an encrypted cover, send it across a public medium, and unwrap it only at trusted endpoint. The history of VPN goes back in the mid-1990s during the development of Point-to-Point Tunneling Protocol led my Microsoft consortium. Layer Two Tunneling Protocol (L2TP) followed as an IETF standard combining features of PPTP and Cisco's L2F. Both relied on IPSec for encryption at the network layer and required kernel-level driver support. Internet Protocol Security (IPSec) itself emerged as a comprehensive suite of protocols for authenticating and encrypting IP packets, operating at Layer 3 of the OSI model [1] [2].

The shift towards SSL/TLS based VPN came through the practical observation TLS already existed in every browser, operated above the transport layer, and could traverse firewalls and NAT devices without special configuration. SSL VPNs, as documented in the NIST Guide to SSL VPNs [11] operate above Layer 4 of the OSI model and create application-layer connections that encrypt data between specific endpoints, rather than routing all IP-level traffic [11] [12]. This defines that SSL VPN is not a network layer tunnel, it is an application layer secure channel. However, commercial SSL VPN implementations in practice still assign clients internal IP addresses and grant broad subnet access upon authentication, preserving the same lateral movement exposure as their IPSec predecessors rather than exploiting the per-service access control that the application-layer model technically enables [5].

The security research community addressed this problem through the concept of Zero Trust Architecture. Zero Trust rejects the assumption of implicit trust based on network location and requires that every access request be verified against the specific resource being

accessed, regardless of whether the requester is inside or outside the network perimeter [5]. The BeyondCorp model implemented by Google demonstrated that Zero Trust principles could be applied in a production enterprise environment, enforcing per-application access control through an application-layer proxy rather than a network-layer tunnel [13]. These define that the risk of lateral movement in network layer is not just a theoretical concern but an architectural limitation with a implementable solution. The gap this project addresses is that this solution has not previously been made available in a self-hosted, dependency-free, and financially accessible form.

2.2 Literature Review

The literature on VPN technology is extensive but fractures sharply along disciplinary lines. The literature relevant to this project can be divided into four distinct areas which are the documented security failure modes of existing VPN architectures, the formal study of Zero Trust access control, the cryptographic foundations of TLS-based secure channels, and the authentication mechanisms used to protect credentials at rest and in transit.

The lateral movement risk in Network Layer (Layer 3) VPN architecture is not just theoretical assumption but a pattern documented across multiple high-severity disclosures. MITRE ATT&CK classifies lateral movement as a primary tactic used by adversaries to enter and control remote system in, defining it as the set of techniques that allow an attacker to progressively move through a network after establishing a foothold [14]. This foothold in context of VPN is the virtual IP address and subnet membership granted to every authenticated client. CVE-2019-11510 in Pulse Secure SSL VPN demonstrated the consequences at scale: unauthenticated remote attackers could read arbitrary files from the gateway including active session tokens, which they then used to access the internal network without credentials [6] and CVE-2023-3519 in Citrix NetScaler Gateway allowed unauthenticated remote code execution on the gateway itself, again because the authentication plane and the network access plane were unified in a single reachable process [7]. The CISA advisory AA20-073A documents as organizations use VPNs for telework, more vulnerabilities are being found and targeted by malicious cyber actors [15].

The NIST Special Publication on Zero Trust Architecture provide the definition of Zero Trust Network Architecture, identifying the elimination of implicit trust based on network location as the foundational principle and specifying that every access request must be evaluated against the specific resource being requested [5]. The publication explicitly

identifies the conventional VPN model in which authentication grants network-level access the implicit-trust model that Zero Trust is designed to replace. Ward and Beyer's BeyondCorp paper describes Google's practical implementation of these principles, in which all internal applications are accessed through an application-layer proxy that enforces per-application, per-user authorization at the point of connection rather than at the network perimeter [13]. The BeyondCorp model demonstrates that application-layer access control can replace network-layer tunneling without loss of security or functionality. This project applies the same architectural principle at a scale and cost appropriate for small to medium organizations.

Donenfeld introduced WireGuard as a critique of prior VPN architectures. WireGuard's kernel paper observed that OpenVPN's user-space architecture forces every packet to cross the user/kernel boundary twice, introducing significant overhead, the reported benchmark also showed WireGuard achieving higher throughput than both OpenVPN and IPsec while consuming less CPU. The architectural argument was that a smaller codebase thus reducing the attack surface and simplifies auditing [8]. A performance comparison by Mackey et. al [16] confirmed WireGuard's throughput advantage over OpenVPN in controlled conditions, but also noted that performance differences diminish at lower bandwidth loads typical of enterprise remote-access use cases. This contextualizes Donenfeld's performance claims for the target deployment environment of this project (small-to-medium organizations, academic environments, restricted network contexts), raw throughput is a secondary concern relative to transparency and deployability. WireGuard's architectural argument is sound on its own terms, but it does not address the lateral movement problem where WireGuard still assigns clients network-layer addresses and relies on external firewall rules for access control. Furthermore, WireGuard abandons TLS entirely in favor of the Noise protocol framework and requires kernel module installation, making it unavailable in restricted environments such as shared academic servers, containerized deployments, or platforms without kernel access [8]. This project's application-layer approach resolves both limitations simultaneously.

Dowling et al. (2021) produced a comprehensive formal analysis of the TLS 1.3 handshake using a multi-stage key exchange security model, demonstrating that TLS 1.3's session keys carry their intended security properties under standard cryptographic assumptions and that the protocol's design changes from TLS 1.2 each address concrete weaknesses identified in prior work [17]. This project implements TLS 1.2 or higher which shares the

same core ECDHE key-exchange structure. The formal guarantees that apply to TLS 1.3's ECDHE component also apply to TLS 1.2 when configured with ECDHE cipher suites, as Dowling et al. note that the ECDHE mechanism itself was already secure in TLS 1.2; TLS 1.3's improvements address other aspects of the protocol such as static RSA key exchange and handshake encryption. [17]. The SANS Institute's analysis of TLS 1.2 deployments provides the practitioner counterpoint. TLS 1.2 can be configured securely with ECDHE cipher suites providing forward secrecy, but it is also routinely misconfigured with static RSA key exchange, which provides no forward secrecy and allows retroactive decryption if the server's private key is ever obtained [18]. This project enforces ECDHE through the and prohibits weak cipher suites explicitly, ensuring that the formal security guarantees of the protocol are actually realized in the implementation.

Likewise, authentication in VPN systems is usually separated into two concerns transport authentication and user authentication. The user authentication side carries specific weakness which are not well addressed in mainstream VPN deployments.

PBKDF2 have fundamental architectural limitation which are its very low memory requirements. Unlike bcrypt or scrypt, an attacker with access to ASICs or GPUs can implement PBKDF2 with minimal circuit area and RAM, making parallelized brute-force attacks substantially cheaper than against memory-hard alternatives such as scrypt or Argon2 [19]. The conclusion is that PBKDF2-HMAC-SHA256 at 100,000 iterations, combined with constant-time comparison and brute-force lockout, provides a reasonable layered defence for the core's use case. The FastAPI management API uses bcrypt, a memory-hard algorithm that is more resistant to GPU-accelerated attacks [20]. OpenVPN's community documentation confirms that the separation of authentication mechanisms across protocol layers is standard practice in production VPN deployments [9].

Regrading more on the transparency problem, Mixon-Baca et. al. [10] analyzed 30 VPN application on the Google Play Store and found that major faction of seemingly independent providers was operated by same entities, with shared codebases server infrastructure, and cryptographic materials that users had no means on verifying. These finding demonstrate that the opacity of commercial VPN products is not merely an academic concern but has practical consequences for user trust and security. Unlike the python-vpn reference implementation that was reviewed as prior work [21], this project does not use any external VPN libraries, implements all session state management directly

in pure Python, and additionally provides a complete control plane i.e. user management, session monitoring, and audit logging which the reference implementation doesn't address. ITU X.509 [22] defines the certificate framework this project uses for server identity verification.

Through all these findings, the literature establishes four clear findings that directly motivates this project's design. First, the lateral movement risk of Layer 3 VPN architectures is real, exploited, and architectural rather than incidental [6] [7] [15] [14].]. Second, Zero Trust per-application access control resolves this risk but has not previously been implemented in a self-hosted, zero-cost, dependency-free form accessible to small organizations [5] [13]. Third, TLS 1.2 with ECDHE provides formally verified forward secrecy when correctly configured, and correct configuration requires explicit enforcement rather than reliance on defaults [17]. Fourth, commercial VPN transparency failures have real security consequences, and an auditable, open implementation addresses a genuine need that the market has not served [10]. This project fills the intersection of all four findings.

CHAPTER 3 : SYSTEM ANALYSIS

3.1 System Analysis

For this project, the main focus is the implementation of the custom application layer VPN in Python 3.11 to provide secure remote access to an internal service through a TLS-protected tunnel. The analysis was centered on the functional flow of the system, the security controls required at each stage while also relating to the constraints imposed. The resulting system is intentionally lightweight, transparent, and deployable without kernel-level networking or external cryptographic dependencies.

3.1.1 Requirement Analysis

Requirement analysis identifies the exact services, constraints, and behaviors the system must support. The requirement analysis was carried out by examining the project brief, session lifecycle specification, threat model, and target deployment environment. The system is derived around three user groups: a VPN client operator who authenticates and forwards the application traffic, a VPN server administrator that manages credentials and monitors sessions, and protected internal service that receives forwarded traffic through the tunnel.

i. Functional Requirements

The functional requirements describe what the VPN system is expected to perform during connection, authentication, tunneling, monitoring, and disconnection, each requirement listed below defines how the system actually works.

- **Secure Connection and Authentication**

The VPN client shall connect to the VPN server t port 8443 through TCP and initiate a TLS 1.2 handshake before any application data is exchange and the server will present its certificate during the handshake and the client shall verify the certificate against trusted CA file. If certificate validation fails, the client will reject the connection and further communication. After the TLS channel is established, the client shall provide their username and password inside an encrypted channel. If authentication is successful then the server will generate a unique session token and return a positive response and if it fails the sever returns an error response and deny access.

- **Secure password handling and login control**

The server shall store the password using PBKDF2-HMAC-SHA256 with random salt for each user. The server will then track any failed authentication attempts by IP address and block the IP that exceeds 3 failed attempts within 60 seconds for 300 seconds. During that period, the server will reject any new authentication attempts made by that IP before performing credential verification.

- **Session creation and Time control**

The system will also create a unique session for each successful login and store the client IP, connection timestamp, last activity timestamp, and bytes transferred. Any session that remains inactive for 3600 seconds will be closed automatically and removed from memory. Expired sessions will not be allowed to continue sending or receiving data.

- **Local forwarding and traffic relay**

The client will open local port 9000 after log in and forward application traffic through the VPN tunnel. The server then will connect to the target internal services and relay data bidirectionally between the encrypted client connection and the internal service without altering the payload.

- **Cleanup, logging, and monitoring**

The system closes all the sockets properly when disconnected or catches error, then log all major events in the structured JSON-line format with sensitive data removed, and expose runtime statistics through a monitoring endpoint.

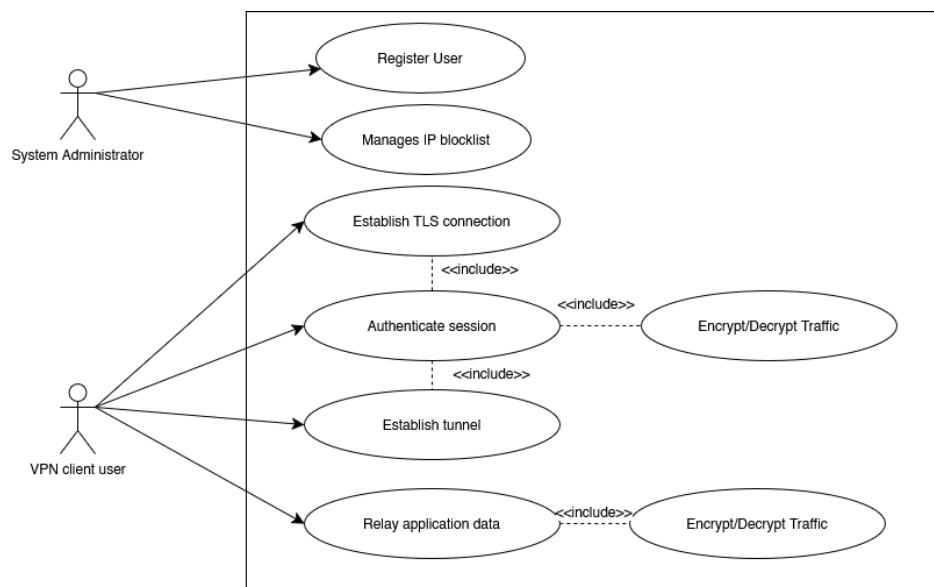


Figure 3. 1: Use Case Diagram of VPN

ii. Non-Functional Requirements

Non-functional requirement focuses on the reliability, performance, portability constraints that the system must satisfy.

- **Security**

All the communication between client and server will use TLS 1.2 or higher which supports Perfect Forward Secrecy, and password comparison will use a constant time matching. Passwords, session tokens, private keys, and other keys will not be stored in plain text in case of accidental exposure of sensitive information through logs, error messages, or monitoring output.

- **Reliability**

The system will close sockets and remove sessions correctly even when errors occur. It will avoid resource leaks during repeated connections, disconnections, and session expiry and it done through structured exception handling.

- **Performance and scalability**

The system shall support at least 50 concurrent sessions without failing session creation or breaking active relays with a buffer to keep forwarding efficient and low-latency in the bidirectional traffic.

- **Portability**

The system shall run on any platform as long as it supports Python 3.11 with OpenSSL and will not require kernel modules, TUN/TAP interfaces, or elevated administrative privileges. This makes implementation suitable for academic and restricted environments.

- **Maintainability and observability**

The system will follow a modular design where each modules have single-responsibility components, use type hints in public interfaces, write logs in JSON-lines format with one event per line to support easy parsing, filtering and audit review, and keep the monitoring endpoint available during normal operation with monitoring features that will not interfere with active VPN sessions.

3.1.2 Feasibility Analysis

i. Technical Feasibility

The project is technically feasible since all the required needs are available within the Python's standard libraries. Python's 'ssl' module handles secure connection using trusted

OpenSSL technology for things like key exchange, certificate checks, and secure context setup. The ‘socket’ module handles internet connections and traffic forwarding and ‘hashlib’ securely scrambles passwords with PBKDF2-HMAC-SHA256, while ‘hmac’ ensures safe password comparisons. ‘thread’ and ‘select’ handles multiple users without blocking with proper efficiency. Certificate are created using the free OpenSSL command-line tools. The VPN itself runs in app level with no special tweaks, virtual network cards, and admin rights making it easy to run on laptops, or school servers where there is no need for advanced setups.

ii. Operational Feasibility

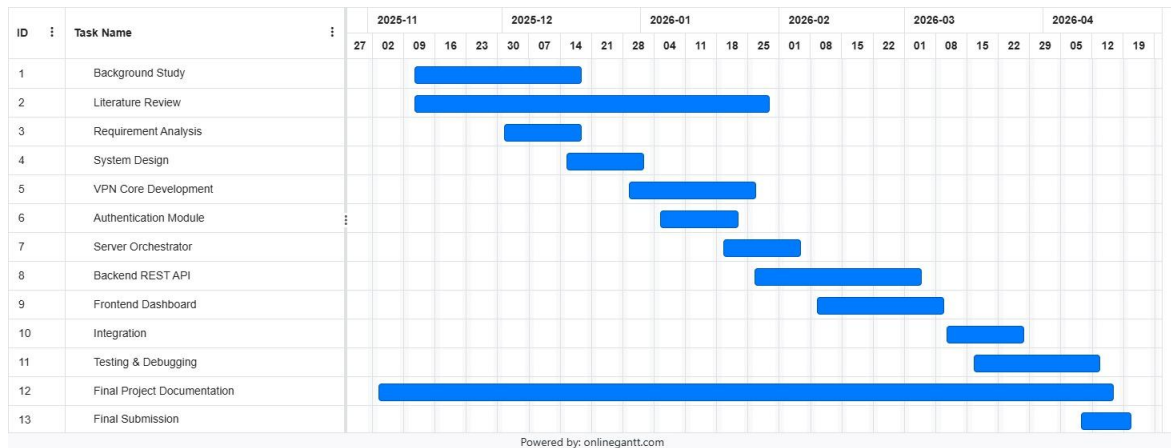
The system is operationally feasible since it is designed for deployment by organization or individual with standard Python environments. The server can be launched using command from the project root directory, and generate certificates using simple OpenSSL commands. The client operates as a lightweight Python process that does not modify system-level routing tables or require administrative privileges. The built in SOC monitoring dashboard provides a real-time stat, and log saves everything in a simple JSON format for easy review. The operational overhead of running this system is minimum, making it suitable for small to medium organizational environments.

iii. Economic Feasibility

This project cost nothing extra and is economically feasible due to no licensing costs, no third-party service fee and no infrastructure-specific dependencies. The project uses Python 3.11 which is free, and for certificate generation OpenSSL is used which is freely distributed, and is open source. The only possible cost is the team's time, already covered in the semester schedule. Compared to commercial VPN service that requires the subscription costs or open-source solutions that require expensive security expertise to configure correctly, this solution provides a cost-effective, self-hosted alternative for organizations that prioritize control and transparency.

iv. Schedule Feasibility

The project is scheduled feasible within the timeframe adopting the Waterfall Model for the clearly defined boundary phases allowing for timely completion within the allotted development period.



3.1.3 Object-Oriented Analysis

The system is modelled using an object-oriented approach and analysis includes class and object diagram to showcase the structural relationships, sequence diagrams to capture dynamic interaction flows, state diagrams to show the session lifecycle transitions, and activity diagrams to define process flows.

a. Object modelling using Class and Object Diagrams

- **Class Diagram**

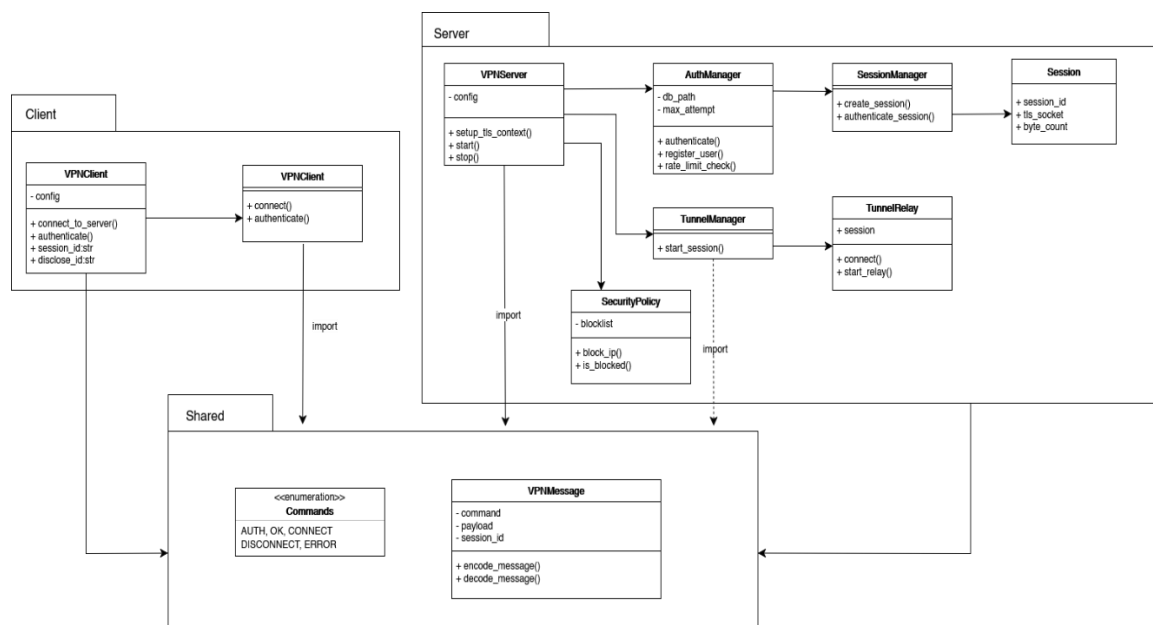


Figure 3.3 Class Diagram of VPN system

This diagram shows the static structure of the system where all the major classes, their key attributes and methods and the relation between them. It shows VPN Message to

hold the commands, data dictionary, and exposes `encode_message()` and `decode_message` methods. Command includes all the valid protocol tokens. The VPNServer sets up secure connections, starts/stops, handles incoming clients and works with AuthManager to check logins and tracks failed attempts, and SecurityPolicy that blocks bad IPs. The session class tracks each user's connection, SessionManager oversees all session, and TunnelRelay forwards data between user and internet. VPNClient connects to the server, logs in, starts local proxy.

- **Object Diagram**

The object diagram shows the system at a particular moment in time, showing how different components interact and relate to each other. The instance `client1` of VPNClient is connected from host 127.0.0.1, using local port 9000, and its session is marked as active. The instance `server1` of VPNServer is listening on port 8443, operating under TLS version 1.2, and currently tracking one active session. The instance `service1` of InternalService is running on localhost at port 8080 and its status is running.

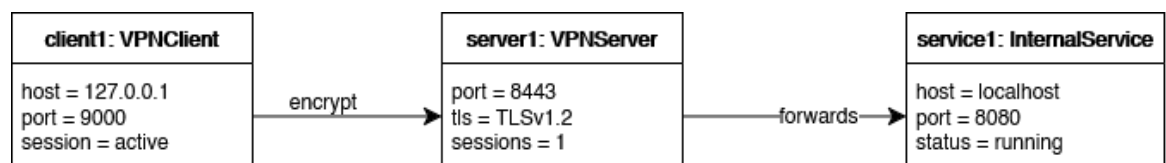


Figure 3. 4: Object Diagram of VPN System

b. Dynamic modelling using State and Sequence Diagrams

- **State Diagram**

The state diagram shows the different conditions a VPN session can be in and the events that cause it to move from one condition to another. It captures the lifecycle of a single session from the moment the client initiates a connection to the moment it ends.

The session begins in the Connecting state, where the client and server perform the TLS handshake and the server verifies the client's certificate. When the handshake succeeds, the session transitions to the Authenticating state, where the server checks the user's credentials. If authentication fails, the session loops back, and after a defined number of failures the IP is blocked. If authentication succeeds, the session moves into the Active state, where encrypted data is relayed bidirectionally between the client and the internal service. The session remains in the Active state until either

the client sends a DISCONNECT command, the connection drops, or the session timer expires. At that point the session transitions to the Disconnected state, where all sockets are closed, resources are released, and the event is written to the log.

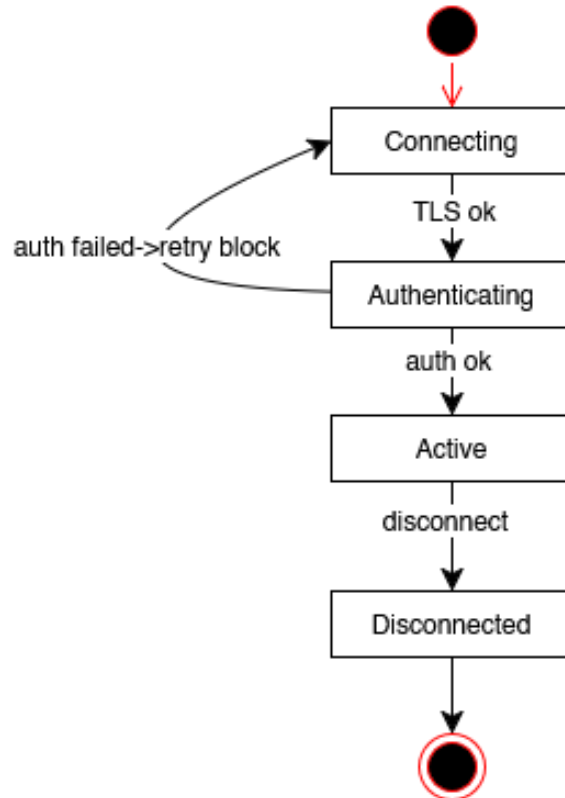


Figure 3. 5: State Diagram of VPN System

- **Sequence Diagrams**

This diagram illustrates that the session begins with the client sending a TLS connect request to the server on port 8443. The server completes the TLS handshake and sends back confirmation. The client then sends an AUTH message containing the username and password, encrypted inside the TLS channel. The server verifies the credentials and responds with an OK message along with a unique session token. The client then sends a CONNECT message specifying the target host and port of the internal service. The server opens a plain TCP connection to that internal service. With the tunnel fully established, the client sends DATA messages containing the encrypted payload. The server decrypts each message, forwards the raw data to the internal service over TCP, receives the response, encrypts it, and sends it back to the client as a DATA message. When the session is complete, the client sends a DISCONNECT message, and the server closes both connections and cleans up the session.

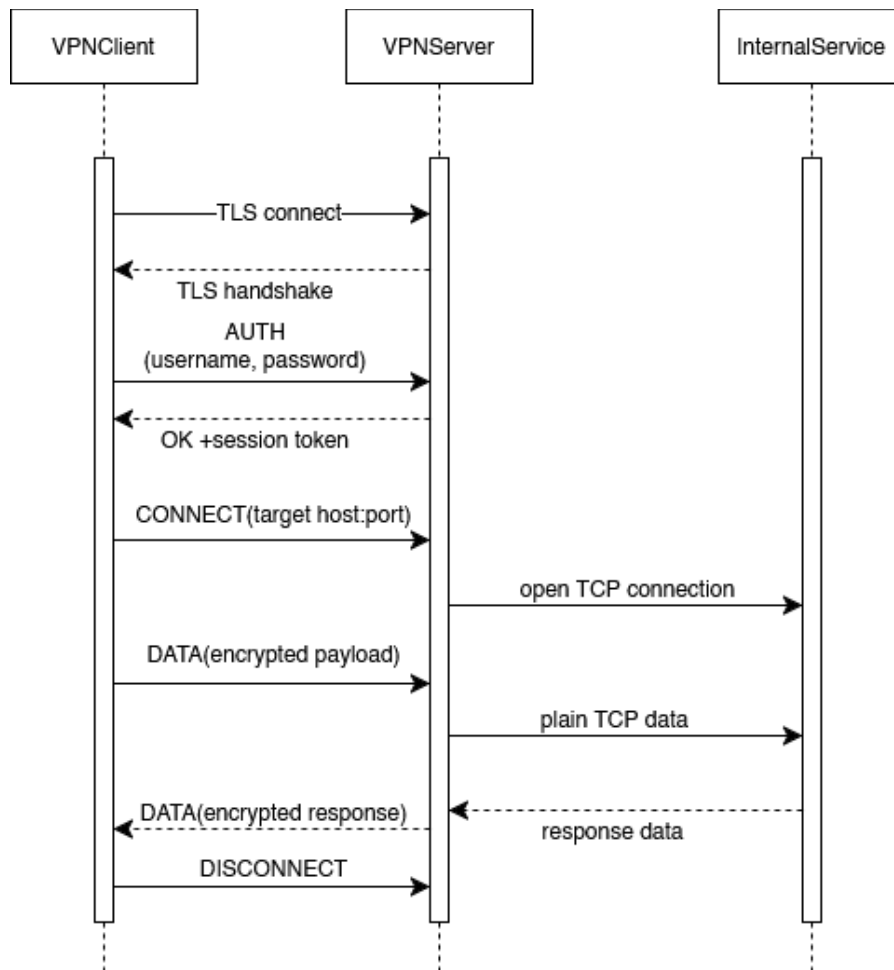


Figure 3. 6: Sequence Diagram of the VPN system

c. Process modelling using Activity Diagrams

The activity diagrams show the workflow of the action takes place during a VPN session, from the initial connection through to session teardown. It focuses on what happens rather than who is doing it. The flow begins when the client connects to the server. The first action is the TLS handshake and certificate verification, which establishes the encrypted channel. Once the channel is secure, the client sends its credentials in an AUTH message. The diagram then reaches a decision point: if authentication fails, the session is rejected and the IP may be blocked, ending the flow on the failure path. If authentication succeeds, the flow continues to tunnel establishment, where a CONNECT message is sent and the server opens a connection to the internal service. The system then enters the data relay phase, where encrypted DATA messages flow back and forth through the tunnel for as long as the session

is active. When the session ends, the teardown action closes all sockets, removes the session record, and writes the final log entry. The flow then reaches the end state.

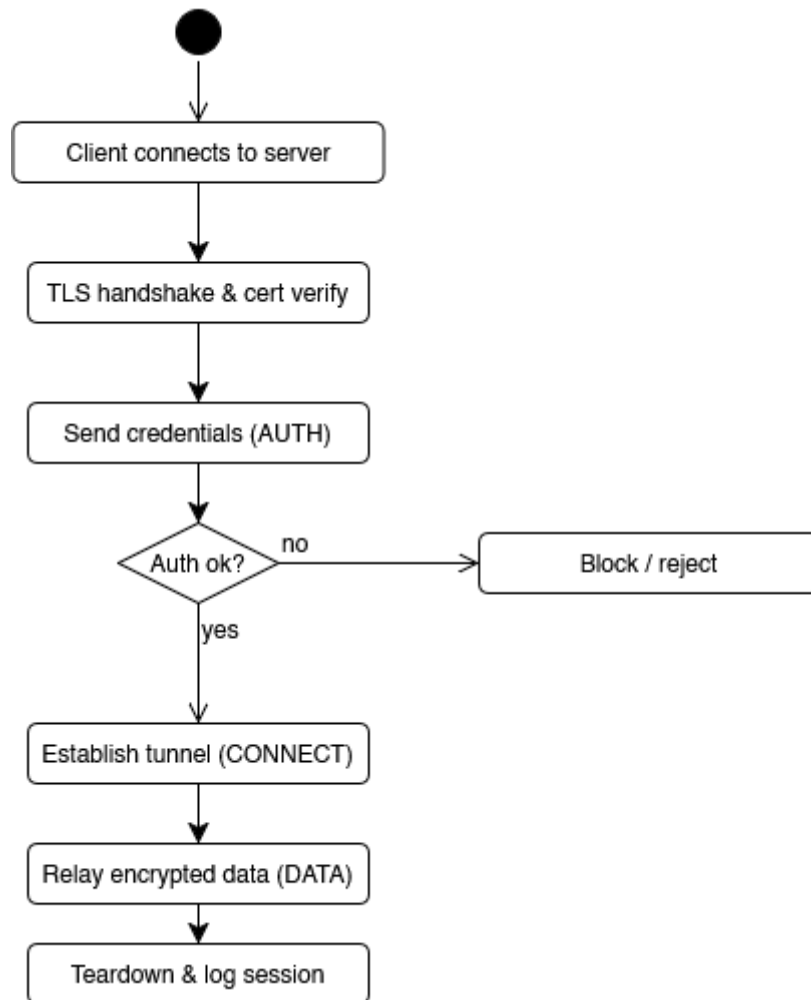


Figure 3. 7: Activity Diagram of VPN System

CHAPTER 4 : SYSTEM DESIGN

4.1 Object Oriented System Design

System design is the blue print to translate the analysis model into a architectural specification. The design refines the class relationships identified in analysis, introduces component and deployment structures.

4.1.1 Refinement of Class diagram

The refinement of the class diagram includes the detailed information about class and their relationship. The refined class organizes the system into three packages server, client, and shared with strict dependency rules. The shared package doesn't have dependency on client or server package but the server package depends on the shared package only and the client package too depends on the shared package only.

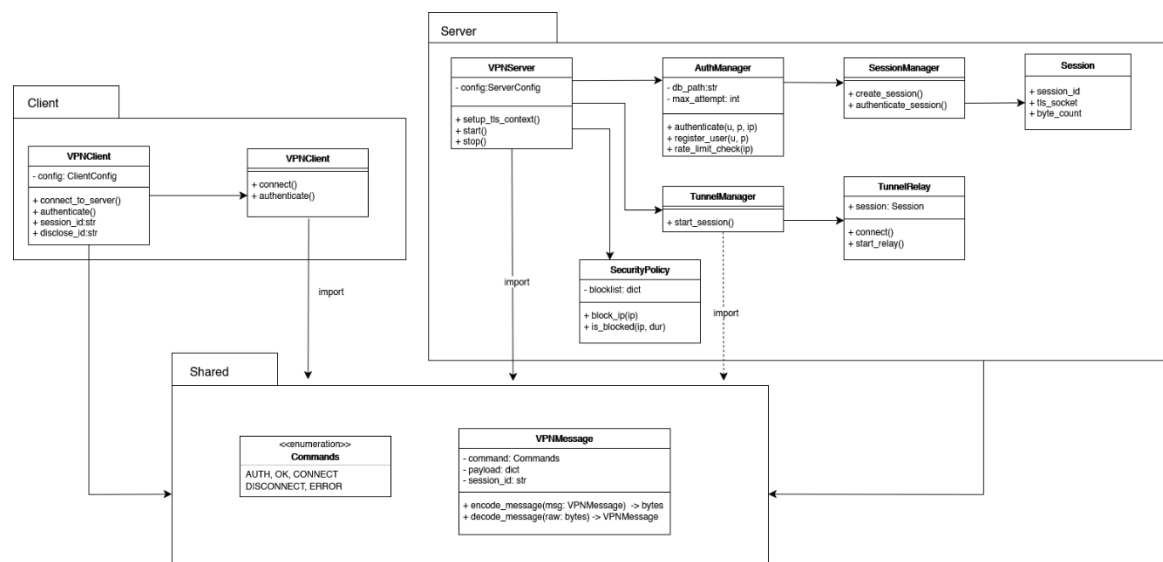


Figure 4. 1: Refinement of Class Diagram of VPN System

This diagram shows the static structure of the system where all the major classes, their key attributes and methods and the relation between them. It shows VPN Message to hold the commands, data dictionary, and exposes `encode_message()` and `decode_message` methods. Command includes all the valid protocol tokens. The **VPNServer** sets up secure connections, starts/stops, handles incoming clients and works with **AuthManager** to check logins and tracks failed attempts, and **SecurityPolicy** that blocks bad IPs. The session class tracks each user's connection, **SessionManager** oversees all session, and **TunnelRelay**

forwards data between user and internet. VPNClient connects to the server, logs in, starts local proxy.

4.1.2 Refinement of Object diagram

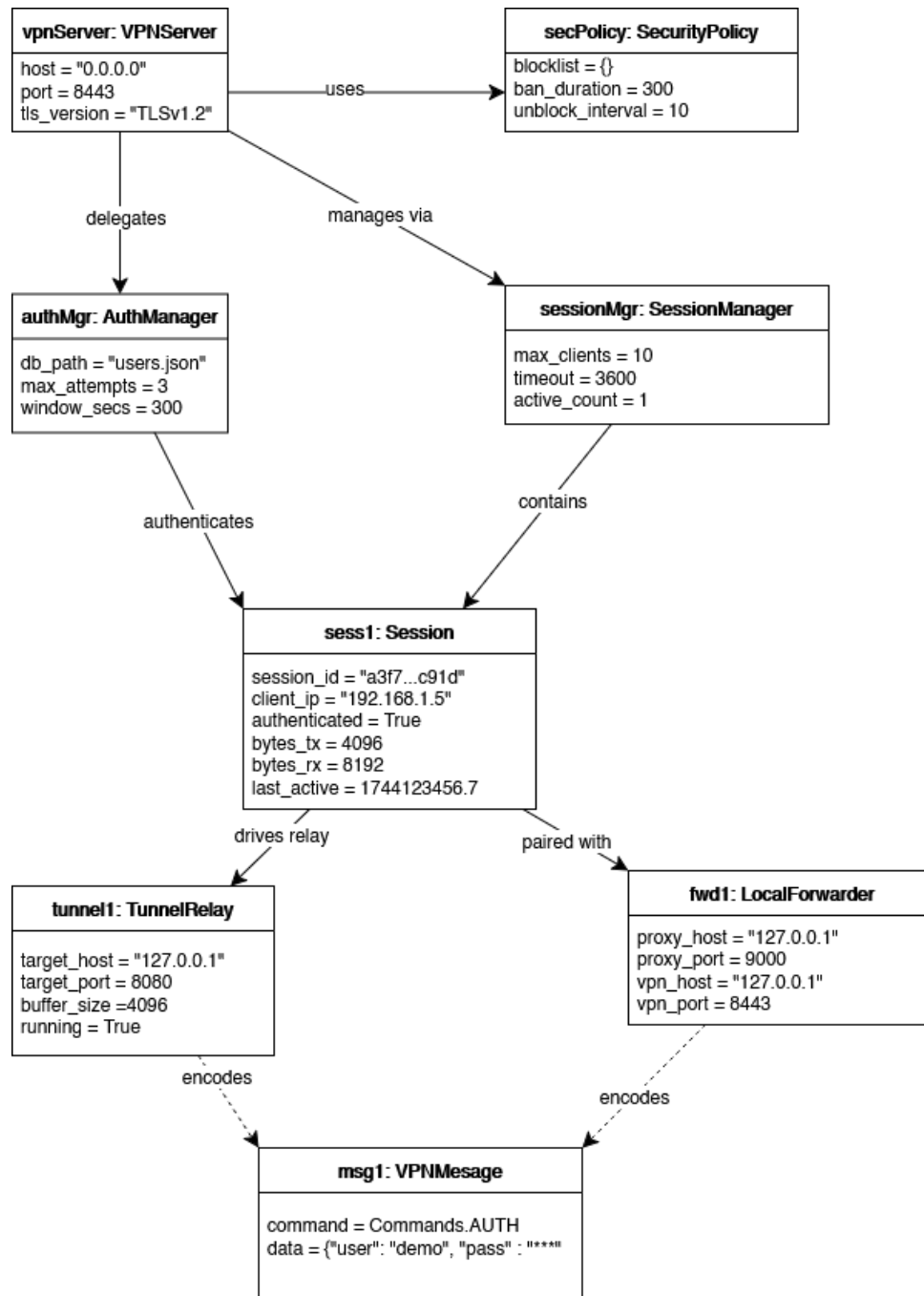


Figure 4. 2: Refinement of Object Diagram of VPN System

The object diagram demonstrates instance of the SSL/TLS VPN system at specific moment when a single client, identified by IP address 192.168.1.5 and username demo, has successfully authenticated and is actively relaying data through an established tunnel.

Unlike the class diagram which shows abstract blueprints, every element in this diagram is a named instance with concrete slot values reflecting the actual state of memory during a live session. The diagram contains seven object instances arranged across four rows, connected by labelled links that show the runtime associations between them.

4.1.3 Refinement of State diagram

The refined state diagram now captures the complete lifecycle of single VPN session from start termination. Initially, it only included the minimum states not seven states are shown in a vertical center column with error exits branching left and right.

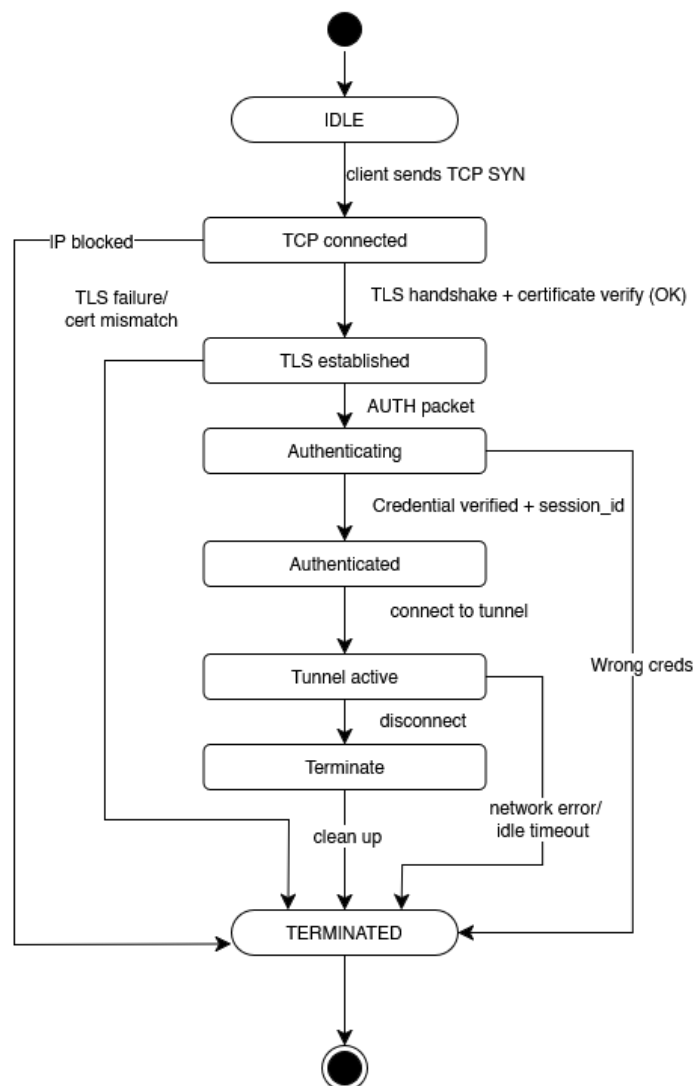


Figure 4. 3: Refinement of State diagram of VPN System

The session begins at the filled circle start pseudo state and enters idle state then transition out when client sends TCP SYN to port 8443. The TCP Connection is formed then proceeds to TLS Established. A dashed error arrow on the left covers the TLS failure and certificate

mismatch case. The transition labelled "AUTH packet received" leads to next state which is Authenticating. It checks recent failed attempt of the specific IP and checks the password. The session now has a unique ID and is fully logged in in Authenticated. Then VPN tunnel runs where data flows both in both way through the encrypted tunnel. Finally in Terminating the server closes both the TLS and TCP connection.

4.1.4 Refinement of Sequence Diagram

The sequence diagram shows the workflow of the VPN in a clear and step-by-step manner. The diagram shows the dynamic message exchange between the participants. The process begins with VPNClient initiating a TLS Handshake on VPNServer lifelines. Once handshake occurs the VPNClient encodes an AUTH VPNMessage containing the username and password and sends it to the VPNServer. The verification of credentials' successful match calls session creation. Then tunnel establishment occurs followed by bidirectional data relay where the VPNClient sends data. In case of idle timeout, the tunnel gets disconnect and then log metrics is recorded.

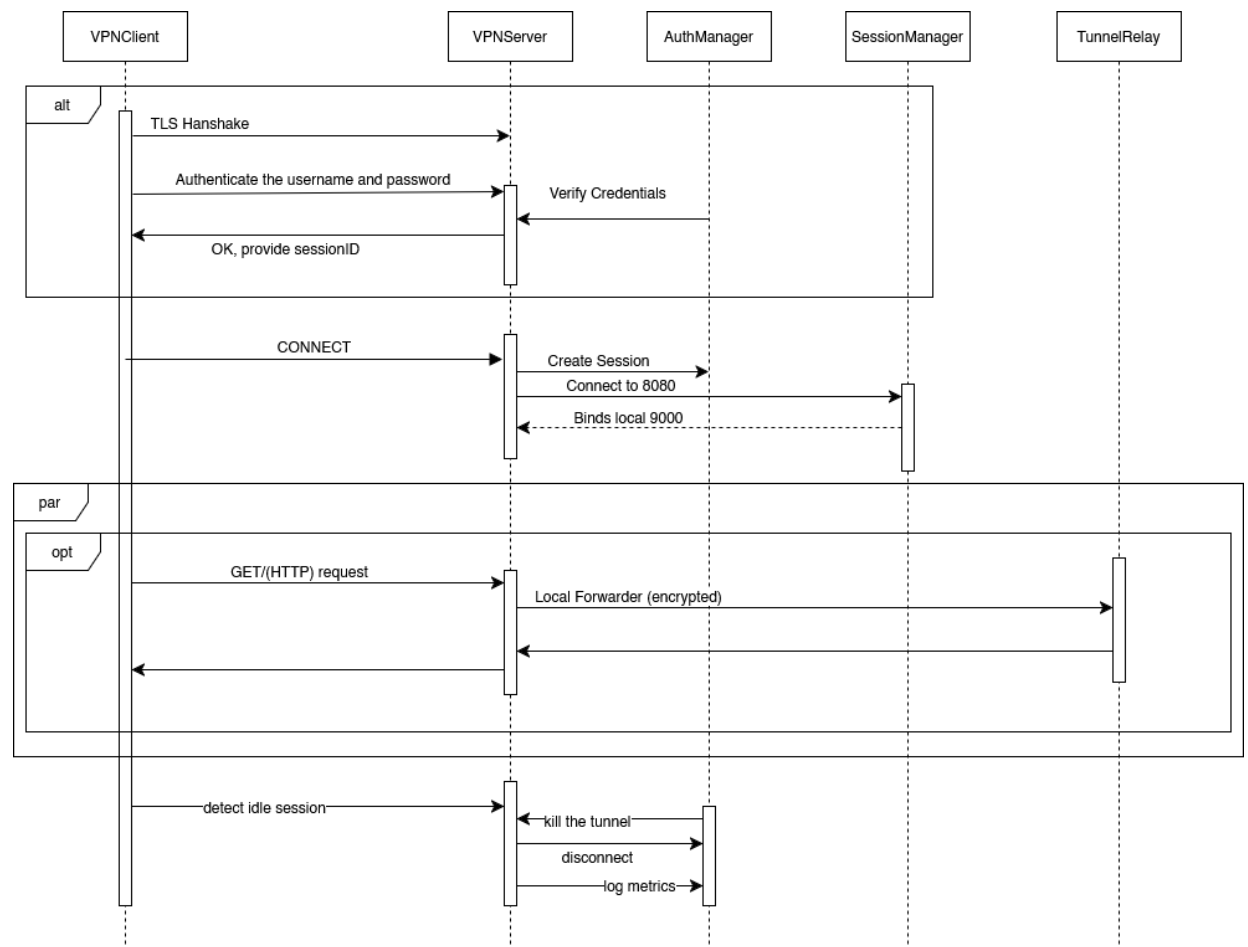


Figure 4. 4: Refinement of Sequence Diagram of VPN System

4.1.5 Refinement of Activity Diagram

The diagram shows the data relay process as concurrent activity divided into two: Client Side on the left, and Server Side on the right. The diagram begins with binding localhost port 9000 followed by Accept App Connection. In parallel, on the server side, connect to 127.0.0.1. A horizontal bar ensures the actual data-forwarding loop only begins once both the local proxy and the upstream connection are ready. After that, both sides enter a “wait for data” state, where they check which socket has incoming data. When the app sends data, the client encrypts it and sends it over TLS to the server, which then decrypts it and forwards it to the real service; the reverse happens for replies from the service back to the app. Both sides keep counters of how many bytes are sent and received. If a read returns empty, both sides move past a second synchronization bar, close all sockets, log the total bytes transferred, clean up the session, and reach the final end state.

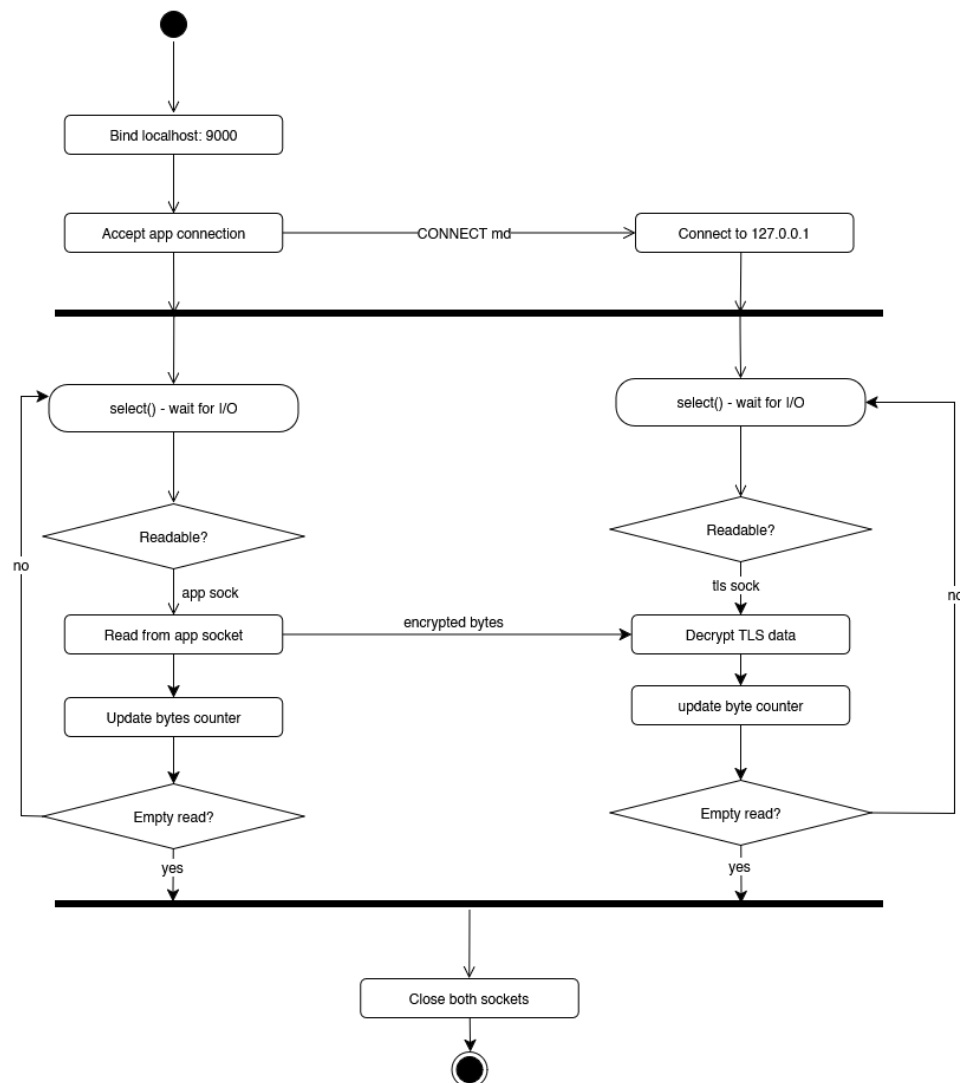


Figure 4. 5: Refinement of Activity Diagram of VPN System

4.1.6 Component Diagram

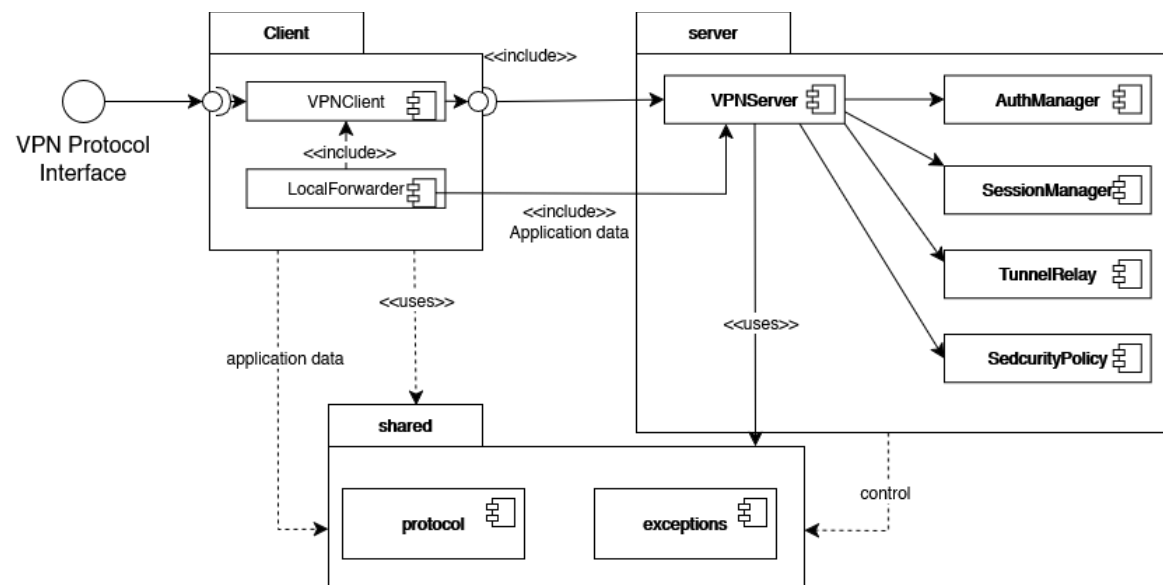


Figure 4. 6: Component Diagram of VPN System

This diagram shows the modular decomposition of the system into packages and components, and the dependency relationships between them. There are three packages shared, client, and server. The shared package consists two components: protocol, which provides the encode and decode operations for VPNMessage frames, and exceptions, which defines the error type hierarchy. The server package consists VPNServer which carries solid association to auth, session, tunnel, and security. Finally, the client package has VPNClient component that carries an association arrow to the forwarder component.

4.1.7 Deployment Diagram

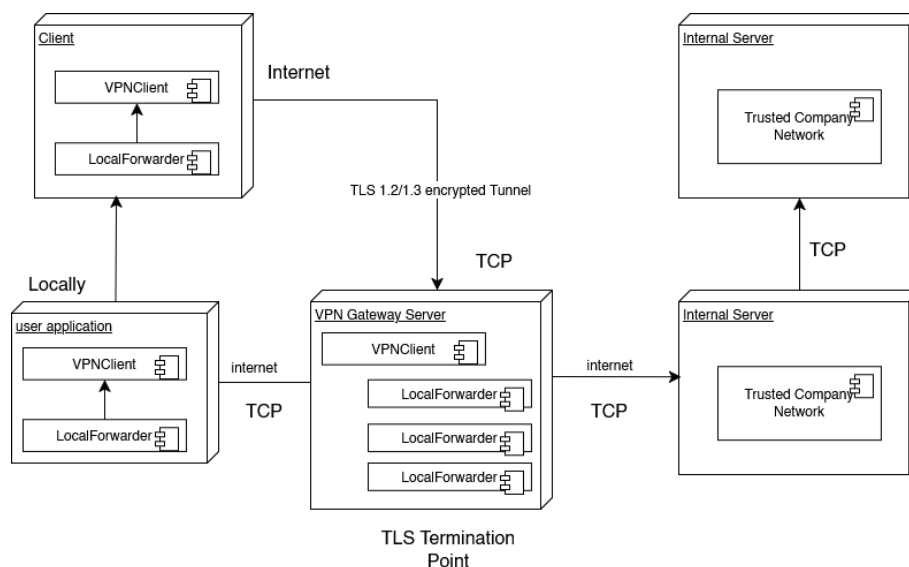


Figure 4. 7: Deployment Diagram of VPN System

The deployment diagram of our system shows the system's physical and logical topology. It illustrates how the VPN software is physically distributed across different hardware environments and how they communicate. It identifies three main nodes: the Client, where the user runs the proxy; the VPN Gateway Server, which sits at the edge of the company network; and the Internal Service, which is the private target (like a database or web app) being protected. The diagram specifically highlights the security boundaries between these nodes, showing that while the connection over the untrusted internet is strictly encrypted using TLS 1.2/1.3, the final hop from the gateway to the internal service uses standard TCP, as it occurs within the trusted corporate network. This visual representation helps engineers understand the physical "path" a data packet takes and where encryption starts and stops.

4.2 Algorithm Details

Since this project uses TLS 1.2 or higher protocol to establish a secure communication channel, the process of establishing a secure communication between the client and server is achieved through combination of multiple underlying cryptographic algorithms to achieve security. In this system, the core algorithms used are ECDHE for key exchange, AES for data encryption, and PBKDF2-HMAC-SHA256 for secure password hashing.

1. ECDHE Key Exchange

During the TLS handshake, the client and server will agree on the shared secret key that will encrypt all message but the problem is how do two parties agree on a secret without the secret travelling across the network. ECDHE solves this problem using a mathematical property of elliptic curves.

Steps involved in this algorithm:

Step 1: Generate Temporary Private Number

Both the client and the server independently generate a temporary private number known only to themselves. This is done to create a secret random number that will be used to compute a public key. This number is never transmitted and is discarded after the handshake.

Mathematically

$$d \in [1, n - 1]$$

Where,

n is the order of base point G

Step 2: Compute Public Value (Public Key)

Each uses a mathematical formula involving their private number and a publicly known elliptic curve to produce a public value, which they share with the other side.

Mathematically,

$$Q = d \times G$$

Where,

Q = Public key (a point on the curve)

d = Private key (random integer from Step 1)

G = Base point (publicly known)

$\times$ = Elliptic curve scalar multiplication

Step 3: Compute Shared Session Key

Each party uses the other's public key and their own private key to compute the same shared secret without ever transmitting it. This is possible due to both party applying same formula using other party's public value and their private number. Due to the mathematical property of elliptic curves, both arrive at same result which is shared session key.

Mathematically,

Client computes:

$$S = d_{client} \times Q_{server}$$

Server computes:

$$S = d_{server} \times Q_{client}$$

Where:

S = Shared secret point (same for both)

d_{client} = Client's private key

d_{server} = Server's private key

Q_{server} = Server's public key (received from server)

Q_{client} = Client's public key (received from client)

Step 4: Discard Private Numbers

Private keys are permanently removed after the shared secret is computed.

Implementation Details:

Input: Two independently generated temporary numbers, one from each side.

Output: A shared session key that both sides hold identically, without it ever being transmitted.

In this system, ECDHE is enabled automatically by Python's ssl module when OP_SINGLE_ECDH_USE is set in the server's SSL context. This flag instructs the system to generate a fresh temporary key pair for every new connection rather than reusing any previous values. The shared key that results is then used by AES to encrypt all subsequent messages for that session.

2. AES Encryption

After the TLS handshake is complete and both sides have a secret key, every message sent through the VPN tunnel must be scrambled so that anyone who intercepts it on the network sees only meaningless characters. AES is the algorithm that makes it possible.

Step 1: Sender takes original message + shared secret key

This step combines the plain text message and the shared secret key as the two inputs required for AES encryption.

Mathematically,

$$E(K, P) = C$$

Where:

E = AES encryption function

K = Shared secret key

P = Plaintext message

C = Ciphertext

Step 2: Block Division + Mathematical Transformations

AES divides the message into fixed-size block and scrambles each block using series of mathematical transformations using the key.

Mathematically,

Divide Message into Fixed-Size Blocks

$$P = P_1 \parallel P_2 \parallel P_3 \parallel \dots \parallel P_m$$

Where:

P = Original plaintext message

P_i = Each block (128 bits)

m = Number of blocks = $\lceil \frac{|P|}{128} \rceil$

$\parallel$ = Concatenation

If last block is shorter than 128 bits:0

$$P'_m = P_m \parallel pad$$

$$pad = (b)(b)\dots(b)$$

Where: $b = 16 - len(P_m)$

Then mathematical transformation on each block

$$C_i = AES - 256 - Encrypt(K, P_i)$$

Where AES-256 applies 14 rounds of transformations.

Step 3: Ciphertext Output + Network Transmission

This step takes in the scrambled output from the AES transformations and prepares it for transmission across the network.

Step 4: Receiver Decryption

The receiver applies the same transformations in reverse using the same key to recover the original message.

Implementation details

Input: A message of any length and the shared session key produced by ECDHE.

Output: The same message in scrambled form, unreadable without the key.

In this system, AES encryption is handled entirely by the TLS layer. When the VPN client sends data through its TLS socket, Python's ssl module automatically encrypts it using AES before it leaves the machine. When the VPN server receives data, the ssl module automatically decrypts it. Neither the client nor the server application code needs to call AES explicitly — it operates transparently underneath the TLS channel.

3. PBKDF2-HMAC-SHA256

When a user register with the VPN system, their password must be store on the server. However, storing passwords in plain text is extremely dangerous thus if the server's storage is ever compromised then even user's password would be exposed immediately. PBKDF2 solves this issue by transforming a password into a fixed-length coded value in a way this is intentionally slow and extremely difficult to reverse.

Step 1: Generate a unique random salt

When a user registers, a unique random value called a salt is generated for them. The salt ensures that even two users with the same password will produce completely different stored values.

$$salt \stackrel{\$}{\leftarrow} \{0, 1\}^{256}$$

Where: $\stackrel{\$}{\leftarrow}$ means “randomly selected from”

Step 2: Combine password + salt and hash repeatedly

The user's password is combined with the salt and passed through the SHA-256 hashing function 100,000 times in sequence. Each pass takes the output of the previous pass as its input.

$$h = PBKDF2(password, salt, iterations = 100000)$$

Step 3: Store hash + Salt on server

The final output which is hash is stored on the sever alongside the salt.

$$stored = (salt, iterations, hash)$$

Step 4: Verify login

When the user later logs in, the same process is applied to the submitted password using the stored salt. If the result matches the stored hash, the password is correct.

$$h_{submitted} = PBKDF2(password_{submitted}, salt_{stored}, iterations)$$

$$is_correct = (h_{submitted} == h_{stored})$$

Implementation Details

Input: The submitted username and password from the login attempt.

Output: The submitted password is correct or not.

The PBKDF2 computation is performed using Python's `hashlib.pbkdf2_hmac` function. The salt is generated using `os.urandom(32)`, which produces 32 cryptographically random bytes guaranteed to be unique. The comparison between the computed hash and the stored hash uses `hmac.compare_digest` rather than a simple equality check.

CHAPTER 5 : IMPLEMENTATION AND TESTING

5.1 Implementation

The system is implemented as a three-tier platform: a custom SSL/TLS application-layer VPN, a FastAPI REST API management layer, and a React administration dashboard.

5.1.1 Tools Used

The development of the VPN system includes involvement of wide range of tools and technologies to ensure flexibility, security, and scalability.

1. Programming Languages

Table 5. 1: Programming Tools used in VPN system

Programming Languages	Description
Python 3.11	Used for the core VPN engine (server, client, authentication, tunneling), certificate generation, and the FastAPI backend REST API.
TypeScript	Used for the frontend management dashboard, providing type safety in a React-based SPA environment
JavaScript (Node.js)	Used during the build process and for the frontend toolchain.

2. Frameworks and Libraries

Table 5. 2: Framework and Libraries used in VPN system

Frameworks and Libraries	Description
FastAPI	A modern Python web framework used to build the backend REST API, providing automatic validation and OpenAPI documentation.
SQLAlchemy ORM	Used for database interaction, providing clean and parameterized queries that prevent SQL injection.
cryptography	Used to generate RSA private keys, x509 certificates, and Certificate Authority (CA) certificates via the gen_certs.py script.

Python ssl module	Used to wrap raw TCP sockets in TLS, enabling AES-256-GCM encrypted communication between VPN client and server.
React + Vite	Used for building the frontend Single Page Application (SPA) dashboard with a modern development toolchain.
Tailwind CSS	Used to style the frontend dashboard with responsive design system.

3. Database

SQLite is used to store user accounts, session records, and audit logs persistently through the backend.

4. Development and Testing Tools

Table 5. 3: Development and Testing Tools used in VPN system

Development and Testing Tools	Description
Git	Version control for managing code changes across all modules.
Pytest	Python testing framework used to run the unit test suite for the VPN engine.
Wireshark	Network packet analyzer used to visually verify that VPN tunnel traffic on port 8443 appears encrypted while decrypted traffic on the target port is readable in plain form.

5.1.2 Implementation Details of Modules

As stated, the system is organized into three tiers, each with own set of modules.

Module 1: Authentication Module

The Authentication Module is responsible for verifying the identity of a connecting VPN client before any tunnel is established. It ensures that only registered and authorized users can access the VPN.

During the authentication module, the VPN client establishes a raw TCP connection to the VPN server on port 8443 and initiates a TLS handshake. The client verifies the server's identity using the CA certificate (ca.crt). Once the TLS tunnel is active, the client sends an authentication message using the custom binary protocol. The message has command code AUTH (byte value 1) and a JSON payload containing the username and password. The server decodes the binary frame, extracts the JSON credentials, and calls the authentication service, which retrieves the stored password hash and salt from the user store (server_users.json or the backend database). The submitted password is hashed using the same salt and compared against the stored hash. If they match, the server assigns a unique UUID session ID to the client and sends an OK response. If they fail, an ERROR message is returned and the connection is closed.

Module 2: Secure Tunnel Establishment Module

This module is responsible for creating and maintaining a secure, encrypted communication channel between the VPN client and the VPN server.

As for how it works, before the server starts, the certificate generation script (gen_certs.py) creates an RSA private key and a CA-signed x509 certificate. These form the server's digital identity. When the server starts, the setup_tls_context() function loads the server certificate and key, configures cipher suites (HIGH:!aNULL:!MD5:!RC4), and enforces TLS 1.2 as the minimum protocol version. When a client connects, the raw TCP socket is wrapped using context.wrap_socket(). This triggers the TLS handshake, during which the client verifies the server's certificate using the shared CA certificate. As part of the TLS handshake, Diffie-Hellman (DH) key exchange is used to negotiate a temporary session key. The flags OP_SINGLE_DH_USE and OP_SINGLE_ECDH_USE ensure a fresh key is generated per connection, implementing Perfect Forward Secrecy (PFS). All subsequent data sent through the TLS-wrapped socket is automatically encrypted with AES-256-GCM by the Python ssl library, without requiring explicit encryption calls in the application code.

Module 3: Data Transmission Module

This module handles the reliable forwarding of application data between the local client application and the remote target service through the VPN tunnel. It implements the custom binary framing protocol to organize data into structured messages.

During the Data Transmission Module, The VPN client starts a local TCP listener on a configured local port. The client application connects to this local port as if it were the real target service. When the application sends data to the local port, the forwarder reads it and packages it as a DATA command using the binary framing protocol. The frame includes a 4-byte length field, a 1-byte command code, a 36-byte session ID, and the raw payload. The encoded frame is sent over the TLS socket to the VPN server. The server decodes the frame, extracts the target host and port from the CONNECT request, and connects to the real target service. The TunnelRelay on the server side uses `select.select()` to monitor both the VPN socket and the internal service socket simultaneously. When data arrives from either direction, it is forwarded to the other side. After data exchange completes, relay metrics (bytes sent and received) are logged. In persistent mode, the session remains open to accept additional CONNECT requests without re-authentication.

Module 4: Monitoring Module

The Monitoring Module provides visibility into the operational state of the VPN system. It enables administrators to view active sessions, traffic metrics, and system health through both a local diagnostics endpoint and a web-based management dashboard.

Similarly, When the VPN server starts, it launches a lightweight HTTP monitor endpoint at `http://127.0.0.1:9999/stats`. Access to this endpoint requires a control secret token, preventing unauthorized access. Each authentication and tunnel event is logged in JSON format to `vpn.log` with a timestamp, log level, event name, and structured data. This includes connection events, authentication results, relay metrics, and push notification attempts. After each tunnel session, the server sends a push notification to the backend API (`POST /api/sessions/push`) with session ID, username, client IP, and byte counts. The notifier uses exponential backoff retry (1s, 2s, 4s over 3 attempts) if the backend is unavailable. The frontend management dashboard reads these records from the backend API and displays live session data, traffic trends, and user activity summaries.

Module 5: Connection Termination Module

The Connection Termination Module handles the clean shutdown of VPN sessions, ensuring that all resources are released, metrics are saved, and both the client and server reach a consistent closed state.

Finally, in connection termination module, termination can be initiated by the client, by the server, or automatically on an unrecoverable error. Upon receiving a DISCONNECT command, the server stops the relay loop, closes the internal service socket, and logs the final relay metrics. In persistent mode, the server loops back to await the next CONNECT command from the same authenticated session. In one-shot mode, the server proceeds directly to cleanup. The cleanup function removes the session from the SessionManager, sends a final END push notification to the backend API, and closes the TLS socket. The client side detects the closed connection through an EOF signal or explicit DISCONNECT message and shuts down the local listener.

5.2 Testing

5.2.1 Test Cases for Unit Testing

Unit testing was performed using the Pytest framework to validate the behavior of individual components in isolation. The following table presents the key test cases executed.

Table 5. 4: Table for test cases of Unit Testing

TC	Module	Input	Expected Output	Actual Output	Result
U01	Authentication	Valid username and correct password hash	Authentication success; session ID returned	Session ID UUID assigned and returned	Pass
U02	Authentication	Valid username and incorrect password	Authentication failure; ERROR response	ERROR message sent; connection closed	Pass
U03	Protocol (Encode)	VPNMessage with DATA command and payload	Binary frame with correct length prefix	Correct binary frame produced	Pass

U04	Protocol (Decode)	Truncated or malformed binary frame	Protocol error or exception raised	Exception raised on invalid input	Pass
U05	Notifier	Backend unavailable; push START event	3 retry attempts with exponential backoff	3 warnings logged; permanent failure recorded	Pass
U06	TLS Context Setup	Load valid server.crt and server.key	TLS context created with TLSv1.2+	TLS context confirmed with correct settings	Pass

5.2.2 Test Cases for System Testing

System testing was performed by running all components together (backend, VPN server, VPN client, and target service) and verifying end-to-end behavior across real operational scenarios.

Table 5. 5: : Table for test cases of System Testing

TC	Scenario	Steps / Input	Expected Result	Actual Result	Result
S01	Successful VPN Connection	Start backend, VPN server, and client with valid credentials and service config	TLS tunnel established; session ID assigned; 'VPN tunnel ready' message shown	Tunnel established successfully; session logged in vpn.log	Pass
S02	Invalid Login Attempt	Start client with wrong password	Authentication failure; connection refused; no session created	ERROR response sent; connection closed cleanly	Pass
S03	Secure Data Transmission	Send HTTP request through VPN tunnel;	Port 8443 shows 'Application Data'	Confirmed via Wireshark: encrypted on	Pass

		capture traffic with Wireshark on port 8443	(encrypted); target port shows plain data	8443, readable on 9000	
S04	Persistent Multi-Connection	Connect client; send two sequential curl requests on the same session	Both requests tunneled under same session ID without re-authentication	Two relay cycles logged under the same session ID	Pass
S05	Backend Unavailable	Start VPN server and client without starting backend	Push notifications fail with retry; VPN tunnel still functions normally	3 push failures logged per session; tunnel operates unaffected	Pass
S06	Missing Certificate Files	Start VPN server with cert files absent	CRITICAL error logged; server exits immediately	CRITICAL [Errno 2] No such file logged; shutdown initiated	Pass

5.3 Result Analysis

During both unit and system testing, all five modules of the VPN system performed as expected. The authentication module correctly validates the credentials and rejects unauthorized access. The secure tunnel module established TLS-encrypted connections consistently, with Wireshark analysis confirming that all traffic on port 8443 appeared as opaque encrypted Application Data. Data transmission was reliable across multiple sequential connections under the persistent session model.

Taking account to security observation, the test cases verified that the TLS context is successfully initialized with a minimum protocol version of TLS 1.2 and strong cipher suites (AES-256-GCM), confirming that weak legacy algorithms such as SSLv3 and RC4 are excluded by configuration. It also confirmed that the OP_SINGLE_DH_USE and

OP_SINGLE_ECDH_USE flags are correctly applied, ensuring Perfect Forward Secrecy (PFS). This means that even if the server private key is compromised in the future, past recorded sessions cannot be decrypted. The authentication unit test validates the PBKDF2-HMAC-SHA256 password hashing mechanism. The correct hash passes authentication while an incorrect one is rejected, demonstrating that stored credentials are protected against brute-force and dictionary attacks. Similarly, system-level testing provided direct Wireshark-based evidence that all traffic on port 8443 appears as opaque 'Application Data', confirming that no plaintext leaks through the TLS tunnel. Traffic at the target port was readable only after decryption by the VPN server. It also confirmed that the server-side permission check fails when the backend is unreachable, push notifications fail but the VPN tunnel continues to function, and no unauthorized access is granted. However, no test case currently covers the monitor endpoint secret, which remains a default value and represents an untested security gap.

For performance observation it can be seen that the system testing produced real relay metrics from vpn.log, recording two sequential tunnel cycles under the same session ID (ac123212...) with data transfers of 88+182 bytes and 78+199 bytes respectively. These figures confirm that the tunnel overhead for small API payloads is minimal. It also measured that the real-world impact of exponential backoff retry logic: when the backend was unavailable. The test also implicitly validates multi-threading correctness: two relay cycles completed sequentially within the same session thread without deadlock or data corruption, confirming that the threading model handles repeated use correctly

CHAPTER 6 : CONCLUSION

6.1 Conclusion

The project defines the successful design and implementation of custom SSL/TLS-based VPN system from foundational principles, while demonstrating how secure, and encrypted network tunnels can be constructed at the application layer using open standards and modern cryptographic practices. The system defines a clearly defined problem i.e. enabling secure remote access to internal services over an untrusted network while maintaining data confidentiality, session integrity, and user accountability. The solution achieves this through a layered architecture in which each component has a well-defined responsibility.

The certificate-based PKI infrastructure establishes cryptographic trust between clients and the server. The TL tunnel ensures all the data in transit is encrypted. The binary framing protocol provides a reliable, structured message format for client-server communication. The data forwarding engine transparently proxies any TCP-based application through the encrypted channel without modification to the application itself. The backend REST API and frontend dashboard provide a centralized management plane for monitoring sessions and administering users. The key strength of this project is protocol-agnostic tunneling, perfect forward secrecy, persistent session model, educational transparency, and capability of auditing.

Overall, the project demonstrates that a functional, educationally transparent, and cryptographically sound VPN can be built at the application layer using standard Python libraries, without reliance on pre-built VPN frameworks. The implementation has been verified to work correctly across all five core modules through both unit and system testing.

6.2 Future Recommendation

While the current system meets its core objectives, several areas of improvement have been identified for future versions of the project such as:

- Auto generate cryptographically random secret at server startup and load it from a secured configuration file, preventing unauthorized access to the diagnostics endpoint.
- Implementation of proper GUI client rather than limiting it to command line to increase accessibility for non-technical users.

- Addition of DTLS-based UDP tunneling for latency-sensitive applications such as VoIP or video conferencing
- Expand the multi-user scalability and load management by replacing the threading model with an asynchronous I/O approach
- Extension of authentication module to support enterprise identity standards such as LDAP, Active Directory, etc for real world deployment

REFERENCES

- [1] H. Y. Xu, "The Research of Building VPN Based on IPsec and MPLS Technolog," *Springer's Communications in Computer Information Science*, vol. 164, 2011.
- [2] D. B. I. R. K. Karuna Jyothi, "Study on Virtual Private Network (VPN), VPN's Protocols And Security," *International Journal of Scientific Research in Computer Science, Engineering and Information Technology*, vol. 3, no. 5, 2018.
- [3] T. Mohite, "The importance of VPN in network security," Scalefusion Blog, 1 August 2022. [Online]. Available: <https://blog.scalefusion.com/importance-of-vpn-in-network-security/#what-is-a-vpn>. [Accessed 26 March 2026].
- [4] Z. DeMeyer, "VPN Vulnerabilities Rising in 2025: Are Your Networks Safe?," todyl, 30 September 2025. [Online]. Available: <https://www.todyl.com/blog/vpn-vulnerability>. [Accessed 26 March 2026].
- [5] O. B. S. M. S. C. Scott Rose, "Zero Trust Architecture," National Institute of Standards and Technology, 2020.
- [6] N. I. o. S. a. Technology, "CVE-2019-11510 Detail," National Vulnerability Database, 2019. [Online]. Available: 2019. [Accessed 18 April 2026].
- [7] N. I. o. S. a. Technology, "CVE-2023-3519 Detail," National Vulnerability Database, 2023. [Online]. [Accessed 18 April 2026].
- [8] J. A. Donenfeld, "WireGuard: Next Generation Kernel Network Tunnel," 2017.
- [9] OpenVPN, "Community Documentation," [Online]. Available: <https://openvpn.net/community-docs/?lang=en>. [Accessed 1 March 2026].
- [10] D. J. K. D. J. R. C. Benjamin Mixon-Baca, "VPN Transparency report 2025: Who owns, operates, and develops your VPN matters," 2025.
- [11] P. H. A. O. R. P. Sheila Frankel, "Guide to SSL VPNs," NIST, 2008.

- [12] J. Burke, "IPSec vs SSL VPNs: What are the differences?," TechTarget, 4 April 2025. [Online]. Available: <https://www.techtarget.com/searchsecurity/tip/IPSec-VPN-vs-SSL-VPN-Comparing-respective-VPN-security-risks>. [Accessed 26 March 2026].
- [13] B. B. Rory Ward, "A new approach to enterprise security," vol. 39, no. 6, pp. 6-11, 2014.
- [14] M. Corporation, "Lateral Movement (TA0008)," MITRE ATT&CK Enterprise Framework, 17 October 2018. [Online]. Available: <https://attack.mitre.org/tactics/TA0008/>. [Accessed 18 April 2026].
- [15] C. a. I. S. A. (CISA), "Alert AA20-073A: Enterprise VPN Security," 20 April 2020. [Online]. Available: <https://www.cisa.gov/news-events/cybersecurity-advisories/aa20-073a>. [Accessed April 2026].
- [16] I. M. A. N. F. V. Y. C. Seteven Mackey, "A Performance Comparison of WireGuard and OpenVPN," pp. 162-164, 2020.
- [17] M. F. F. G. & D. S. Benjamin Dowling, "A Cryptographic Analysis of the TLS 1.3 Handshake Protocol," *Journal of Cryptography*, 2021.
- [18] A. M. & D. Bissell, "Legacy in Handshake: Understanding TLS 1.2 Prevalence and Its Operational Risks," SANS, 16 Sept 2025. [Online]. Available: <https://www.sans.org/blog/legacy-handshake-understanding-tls-prevalence-operational-risks>. [Accessed 2 April 2026].
- [19] OWASP, "Password Storage Cheat Sheet," OWASP, 2023. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html. [Accessed 2 April 2026].
- [20] OWASP, "OWASP Cheat Sheet Series - Cryptographic Storage," OWASP, [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Cryptographic_Storage_Cheat_Sheet.html. [Accessed 2 April 2026].

- [21] Q. Wenjie, "Github," 16 Feb 2021. [Online]. Available: <https://github.com/qwj/python-vpn>. [Accessed 2 April 2026].
- [22] ITU, "X.509 : Information technology - Open Systems Interconnection - The Directory: Public-key and attribute certificate frameworks," ITU, 21 March 2019. [Online]. Available: <https://www.itu.int/rec/t-rec-x.509>. [Accessed 2 April 2026].
- [23] E. Barker, "Recommendation for Key Management: Part 1 - General," National Institute of Standards and Technology, 2020.

APPENDICES



```
(.venv) PS G:\Studies\7th_sem\project_report\vpn_prototype_v3_A6> python -m _custom_ssl_vpn.server.vpn_server --port 8443
G:\Studies\7th_sem\project_report\vpn_prototype_v3_A6\_custom_ssl_vpn\server\vpn_server.py:149: DeprecationWarning: ssl.OP_NO_SSL*/ssl.OP_NO_TLS*
options are deprecated
  context.options |= flags
{"timestamp": "2026-04-17T22:17:40.288946Z", "level": "INFO", "event": "Monitor listening on http://127.0.0.1:9999/stats", "data": {}}
{"timestamp": "2026-04-17T22:17:40.288946Z", "level": "INFO", "event": "Monitor control secret: default_unsafe_monitor_secret_123", "data": {}}
{"timestamp": "2026-04-17T22:17:40.288946Z", "level": "INFO", "event": "VPN server started and listening for TLS connections.", "data": {"host": "
0.0.0.0", "port": 8443, "tls_version": "TLSv1_2", "status": "Initialized and Listening"}}
```

Server Running



```
(.venv) PS G:\Studies\7th_sem\project_report\vpn_prototype_v3_A6> python -m _custom_ssl_vpn.client.vpn_client --username test.user1 --password adm
in12345 --target-host 127.0.0.1 --target-port 8000 --server-port 8443
2026-04-17 22:21:34,180 - VPNClient - INFO - Connecting to VPN server 127.0.0.1:8443...
2026-04-17 22:21:34,198 - VPNClient - INFO - TLS handshake and connected successfully.
2026-04-17 22:21:34,240 - VPNClient - INFO - Authentication granted.
2026-04-17 22:21:34,240 - LocalForwarder - INFO - Local proxy binding active on 127.0.0.1:9000
VPN tunnel ready (persistent mode). Connect to: 127.0.0.1:9000
Multiple connections supported. Press Ctrl+C to disconnect.
```

Client Communication

```
{"timestamp": "2026-04-17T22:21:34.195002Z", "level": "INFO", "event": "New Client Connection", "data": {"client_ip": "127.0.0.1", "client_port": 55598}}
{"timestamp": "2026-04-17T22:21:34.240218Z", "level": "INFO", "event": "Authentication Success", "data": {"username": "test.user1", "session_id":
"10cac619-275e-44ed-a5f4-a811a74b6bc1"}}
{"timestamp": "2026-04-17T22:23:12.379496Z", "level": "INFO", "event": "Tunnel Open", "data": {"session_id": "7a4e592e-22d4-4d73-b732-a3e65defb0df",
"target_host": "127.0.0.1", "target_port": 8000}}
{"timestamp": "2026-04-17T22:23:17.375275Z", "level": "INFO", "event": "Internal target closed connection. Sending EOF signal.", "data": {}}
{"timestamp": "2026-04-17T22:23:17.375275Z", "level": "INFO", "event": "Relay Metrics Tracked", "data": {"session_id":
"7a4e592e-22d4-4d73-b732-a3e65defb0df", "bytes_client_to_internal": 163, "bytes_internal_to_client": 1076}}
{"timestamp": "2026-04-17T22:23:17.376057Z", "level": "INFO", "event": "Tunnel Closed", "data": {"session_id": "7a4e592e-22d4-4d73-b732-a3e65defb0df"}}
{"timestamp": "2026-04-17T22:23:17.376057Z", "level": "INFO", "event": "Relay finished, waiting for next CONNECT or DISCONNECT.", "data": {"session_id":
"7a4e592e-22d4-4d73-b732-a3e65defb0df"}}
{"timestamp": "2026-04-17T22:23:29.517673Z", "level": "INFO", "event": "Tunnel Open", "data": {"session_id": "7a4e592e-22d4-4d73-b732-a3e65defb0df",
"target_host": "127.0.0.1", "target_port": 8000}}
{"timestamp": "2026-04-17T22:23:34.524975Z", "level": "INFO", "event": "Internal target closed connection. Sending EOF signal.", "data": {}}
{"timestamp": "2026-04-17T22:23:34.525542Z", "level": "INFO", "event": "Relay Metrics Tracked", "data": {"session_id":
"7a4e592e-22d4-4d73-b732-a3e65defb0df", "bytes_client_to_internal": 139, "bytes_internal_to_client": 1076}}
{"timestamp": "2026-04-17T22:23:34.525542Z", "level": "INFO", "event": "Tunnel Closed", "data": {"session_id": "7a4e592e-22d4-4d73-b732-a3e65defb0df"}}
{"timestamp": "2026-04-17T22:23:34.525542Z", "level": "INFO", "event": "Relay finished, waiting for next CONNECT or DISCONNECT.", "data": {"session_id":
"7a4e592e-22d4-4d73-b732-a3e65defb0df"}}
```

Logging output

NAVIGATION

DashboardSessionsUsersServicesAudit LogProfile

adminadmin@vpn.local

Session Monitor

View and manage active VPN sessions

Active onlyRefresh

SESSION ID	USER	STATUS	CLIENT IP	DURATION	↑ UPLOAD	↓ DOWNLOAD	STARTED	ACTIONS
18cac819-275...	test.user1	Active	127.0.0.1	10m	0 B	0 B	Apr 17, 2026	
7a4e592e-22d...	alice	Active	127.0.0.1	35m	465 B	3.2 KB	Apr 17, 2026	
3f9f289d-8da...	alice	Closed	127.0.0.1	48s	0 B	0 B	Apr 17, 2026	
d2cd8917-2ad...	test.user2	Closed	127.0.0.1	23s	145 B	182 B	Apr 16, 2026	
f25d2895-7ff...	test.user2	Closed	127.0.0.1	27s	2.8 KB	3.6 KB	Apr 16, 2026	
bee59aa-43a...	test.user2	Closed	127.0.0.1	23s	435 B	546 B	Apr 16, 2026	
a2b8c765-98d...	test.user2	Closed	127.0.0.1	42s	145 B	182 B	Apr 16, 2026	
b1d9ef22-985...	test.user2	Closed	127.0.0.1	31s	145 B	182 B	Apr 16, 2026	
a9663db6-ea9...	test.user2	Closed	127.0.0.1	3m	978 B	199 B	Apr 16, 2026	
a418ef7e-8a9...	test.user2	Closed	127.0.0.1	2m	165 B	154 B	Apr 16, 2026	

Session monitoring via dashboard

NAVIGATION

DashboardSessionsUsersServicesAudit LogProfile

adminadmin@vpn.local

User Management

Create, edit, and manage VPN user accounts

Search by username or email...RefreshNew User

ID	USERNAME	EMAIL	ROLE	STATUS	CREATED	ACTIONS
1	admin	admin@vpn.local	ADMIN	Active	Apr 15, 2026 2d ago	
4	alice	alice@example.com	USER	Active	Apr 17, 2026 6h ago	
2	test.user1	test.user1@gmail.com	USER	Active	Apr 15, 2026 2d ago	
3	test.user2	test.user2@gmail.com	USER	Active	Apr 15, 2026 2d ago	
4 users total						

NAVIGATION

DashboardSessionsUsersServicesAudit LogProfile

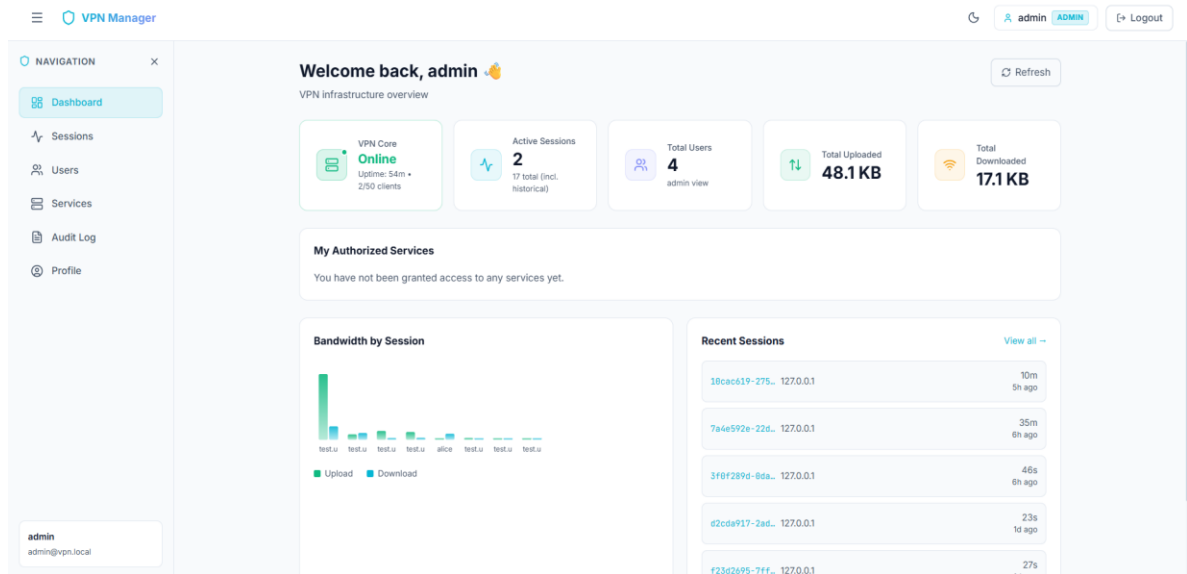
adminadmin@vpn.local

User Management

Create, edit, and manage VPN user accounts

Search by username or email...RefreshNew User

ID	USERNAME	EMAIL	ROLE	STATUS	CREATED	ACTIONS
1	admin	admin@vpn.local	ADMIN	Active	Apr 15, 2026 2d ago	
4	alice	alice@example.com	USER	Active	Apr 17, 2026 6h ago	
2	test.user1	test.user1@gmail.com	USER	Active	Apr 15, 2026 2d ago	
3	test.user2	test.user2@gmail.com	USER	Active	Apr 15, 2026 2d ago	
4 users total						



Dashboard for the VPN manager